Content Hub

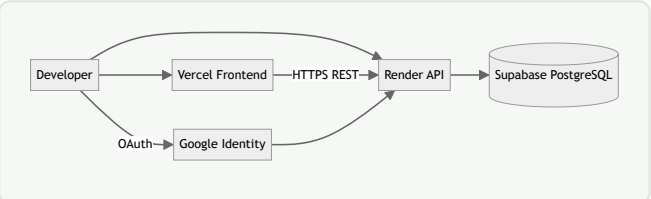
Developer Guide

Version 1.0 · June 2026 · React + Spring Boot + Supabase

Contents

1. Overview

Content Hub is a full-stack blogs and articles platform. The **React** frontend is hosted on **Vercel**; the **Spring Boot** API runs on **Render** (Docker); data lives in **Supabase PostgreSQL**.



2. Tech Stack

Layer	Technology	Version
Frontend	React, React Router, Vite	19.x / 7.x / 8.x
Backend	Spring Boot, Java	3.4.4 / 17
Auth	JWT (jjwt), Google OAuth2 Client	0.12.6
Database	PostgreSQL via JPA + Flyway	17
Hosting	Vercel, Render, Supabase	—

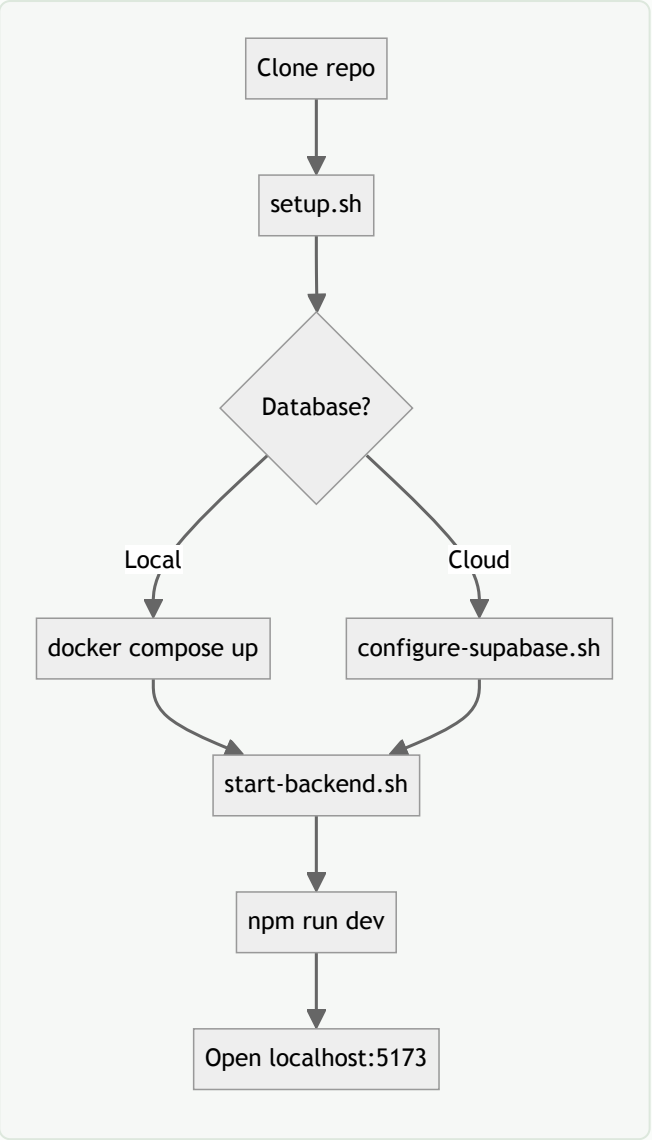
3. Local Development Setup

Prerequisites

- Node.js 20+, Java 17, Docker (optional local Postgres)
- Supabase project or local `docker compose` database

Quick start

```
git clone https://github.com/AbhishekMD1998/content_hub.git
cd content-hub
bash scripts/setup.sh          # installs deps, Maven wrapper
docker compose up -d           # local Postgres on :5433 (optional)
bash scripts/configure-supabase.sh   # OR copy backend/.env.example
bash scripts/start-backend.sh  # API on http://localhost:8080
npm run dev                    # Frontend on http://localhost:5173
```



> In dev, `VITE_API_BASE_URL` is unset — Vite proxies `/api`, `/oauth2`, and `/login` to port 8080.

4. Project Structure

```
content-hub/
├── src/                    # React frontend
│   ├── api/                # HTTP client, auth, blogs, articles
│   ├── components/         # Layout, PostCard, BlogRenderer, etc.
│   ├── context/            # AuthContext, ContentContext
│   └── pages/              # Dashboard, Blogs, Admin, Login
├── backend/                # Spring Boot API
│   └── src/main/java/com/contenthub/
│       ├── config/         # Security, DataSeeder
│       ├── domain/         # User, Blog, Article entities
│       ├── security/       # JWT filter, OAuth handler
│       ├── service/        # Business logic
│       └── web/            # REST controllers
├── scripts/                # Dev & deploy helpers
├── docs/                   # Markdown documentation
├── render.yaml             # Render Blueprint
└── vercel.json             # Vercel SPA config
```

5. API Reference

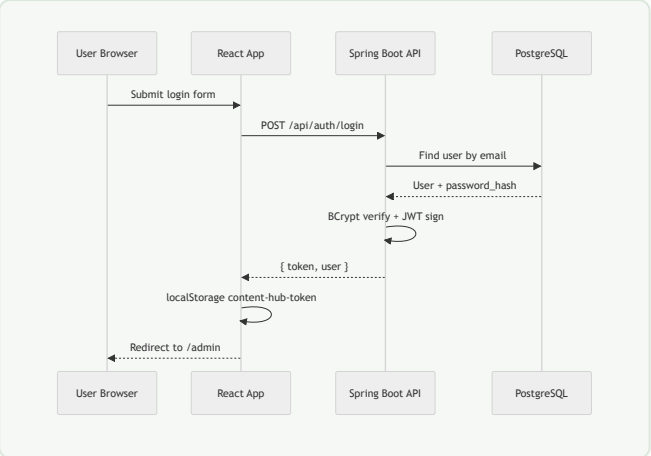
Method	Endpoint	Auth	Description
GET	/api/health	Public	Health check
POST	/api/auth/signup	Public	Register user
POST	/api/auth/login	Public	Email/password login
GET	/api/auth/me	Bearer JWT	Current user profile
GET	/api/auth/config	Public	{ googleEnabled: bool }
GET	/api/articles	Public	List articles
GET	/api/articles/{slug}	Public	Article detail
GET	/api/blogs	Public	List blogs
GET	/api/blogs/{slug}	Public	Blog detail
POST	/api/blogs	JWT	Create blog
DELETE	/api/blogs/{slug}	JWT	Delete blog
GET	/oauth2/authorization/google	Public	Start Google OAuth

Example: Signup request

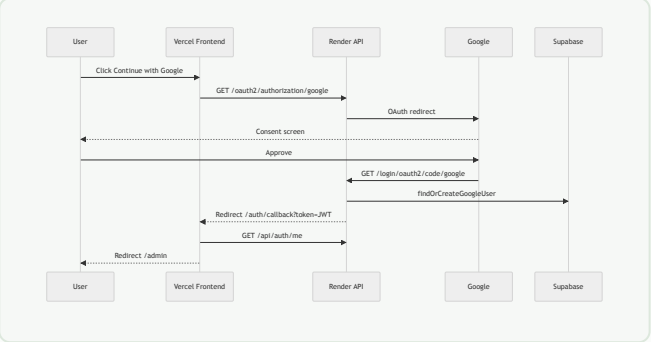
```
POST /api/auth/signup
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "secret123",
  "displayName": "Jane Doe"
}

→ 200 { "token": "eyJ...", "user": { "id", "email", "displayName", "role" } }
```

6. Authentication Flows

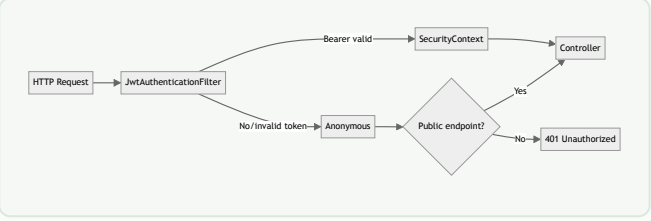
6.1 Email / Password Login



6.2 Google OAuth



6.3 Protected API requests



7. Frontend Guide

Key files

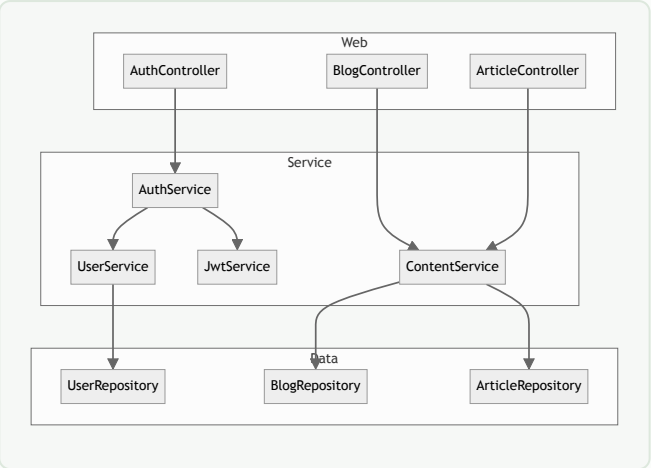
File	Role
src/api/client.js	fetch wrapper, JWT header, error handling
src/api/config.js	VITE_API_BASE_URL → apiUrl()
src/context/AuthContext.jsx	login, signup, logout, user state
src/context/ContentContext.jsx	blogs list, addBlog, deleteBlog
src/components/ProtectedRoute.jsx	Guards /admin route
src/pages/AdminPanel.jsx	Create/delete blog posts

Routing (App.jsx)

Path	Page	Auth
/	Dashboard	Public
/blogs, /blogs/:id	Blogs list & detail	Public
/articles, /articles/:id	Articles list & detail	Public
/admin/login	AdminLogin	Public
/admin	AdminPanel	JWT required
/auth/callback	AuthCallback	OAuth handoff

8. Backend Guide

Layered architecture



Data seeding

`DataSeeder` runs on startup: creates admin user (`admin@contenthub.com` / `admin123`), sample articles, and blogs from `seed/blogs.json` if tables are empty.

9. Environment Variables

Frontend (Vercel)

Variable	Description
VITE_API_BASE_URL	Render API URL, e.g. https://content-hub-api-vkkj.onrender.com

Backend (Render / backend/.env)

Variable	Description
DATABASE_URL / pooler vars	Supabase JDBC connection
DATABASE_PASSWORD	Supabase DB password
JWT_SECRET	JWT signing key (32+ chars)
FRONTEND_URL	Vercel URL for OAuth redirect
CORS_ALLOWED_ORIGINS	Allowed frontend origins
GOOGLE_CLIENT_ID / SECRET	Google OAuth credentials
SUPABASE_POOLER_HOST	Session pooler hostname (IPv4)

10. Deployment

```
# Frontend
npx vercel --prod

# Backend — auto-deploys from main via Render Blueprint (render.yaml)
# Set env vars in Render Dashboard → Environment
```

Production URLs:

- Frontend: https://content-hub-pi-bay.vercel.app
- API: https://content-hub-api-vkkj.onrender.com

11. Troubleshooting

Issue	Fix
Cannot reach API from Vercel	Set VITE_API_BASE_URL on Vercel
Google redirect_uri_mismatch	Add exact Render callback URL in Google Console
Render Network unreachable	Use Supabase session pooler, not direct db.* host
API slow first request	Render free tier cold start (~60–120s)
CORS errors	Match FRONTEND_URL in Render CORS_ALLOWED_ORIGINS